Federal Polytechnic School of Lausanne

Semester project autumn 2025

Master in Computational Science and Engineering

Deep Learning for Prediction of MARFE Plasma Instabilities in Tokamak (TCV)

Author:
Kleon Karapas

Supervisors:
Dr. Yoeri Poels
Dr. Alessandro Pau
Prof. Paolo Ricci

Contents

1 Introduction

In future fusion reactors such as ITER and DEMO, the reliable prediction and control of plasma instabilities is essential to ensure safe and sustained operation. Instabilities can generate severe thermal and electromagnetic loads on plasma-facing components, posing a major challenge for reactor longevity. As a result, developing robust methods to anticipate instabilities and enable timely mitigation is a critical area of fusion research. This project focuses on the MARFE (Multifaceted Asymmetric Radiation from the Edge) instability, characterized by the formation of a cold, dense, and highly radiative region at the plasma edge. Typically originating near the X-point in diverted plasmas, MARFEs can propagate inward, strongly modifying temperature and current density profiles and ultimately triggering instabilities. The study is based on experimental data from the TCV tokamak at the Swiss Plasma Centre (SPC), primarily using the RADCAM radiation camera [10], which provides high-resolution time-resolved measurements of plasma radiation. These data, complemented by additional diagnostics such as magnetic probes and interferometers, form multivariate time series describing the plasma state. The objective of this work is to exploit these time-series diagnostics within a machine learning framework to predict the onset of MARFE instabilities, with the ultimate aim of improving early-warning capabilities for real-time disruption mitigation. In Section 2, we first explore the general structure of the features and define the way in which we label the instabilities for every shot. We then expose in Section 3 the three different models that we compare in this study: a simple feedforward neural network (multilayer perceptron, MLP), a Recurrent Neural Network (RNN) and a Temporal Convolutional Neural Network (TCN). In the final Section 4, we conduct a systematic numerical study of the models, assessing their robustness across instability horizons, their sensitivity to detection threshold tuning, and their temporal alarm behavior under increasingly strict early-alarm penalties. The code used throughout the report is available here: `https://github.com/klekar11/dl_instability`.

Acknowledgments I would like to express my sincere gratitude to Dr. Poels for his continuous support and insightful feedback throughout the course of this project.

2 Data Preprocessing

2.1 Dataset Description and Feature Characteristics

The dataset consists of eight physical features (see Table 1), recorded together with the corresponding measurement timestamps. The sampling interval is $2 \cdot 10^{-4}$s, corresponding to a sampling frequency of 5kHz. Instabilities are labeled per shot, not per time point: a discharge is considered unstable if a MARFE event occurs, and stable otherwise. Two diagnostic events may be reported for each unstable shot: `MARFE_Xpoint` and `MARFE_INmove`. The X-point event generally precedes the INmove event, reflecting the physical evolution of the radiating blob, first forming near the X-point, then moving upward and interacting with the core plasma. Since the INmove phase corresponds more directly to the disruptive behavior, it is used as the effective instability onset time, except in the case where there is only the X-point event for a shot. A total of 135 shots were analyzed. Among them, 69 contain at least one MARFE event and 66 are stable. For 18 shots, the reported MARFE event occurred outside the range covered by the available diagnostics; these shots are excluded. We therefore retain 51 unstable shots for all subsequent analyses.

Table 1: Summary of dataset features and their physical interpretation.

Feature	Short Description
IPLA	Plasma current: represents the total toroidal electric current flowing within the plasma (10^5A).
a_{minor}	Minor radius of the plasma cross-section; shaping parameter defined by magnetic equilibrium: indicates how "large" or "compressed" the plasma cross-section is ($\sim$ 0.2–0.25 m).
δ	Plasma triangularity: describes D-shaping, dimensionless and typically very stable. Positive triangularity corresponds to the plasma corner being shifted towards the high field side (inner part of the vessel) (~ -0.2–0.6).
κ	Plasma elongation: ratio (dimensionless) of the vertical to horizontal extent of the plasma cross-section ($\sim$ 1–2.2).
fir_lids_core	Line-Integrated Electron Density: a chord-integrated measurement of electron density through a line-of-sight across the core. Used as a proxy for plasma density (typically $\sim 10^{19}\text{m}^{-2}$).
sxrcore	Soft X-ray emission from plasma core: encodes impurity radiation and high-frequency MHD activity ($\sim 10^{-1}$–$10^{2}\text{W} \cdot \text{m}^{-1}$).
PradTot	Total radiated power from bolometry: reflects impurity accumulation and radiative losses ($\sim$ 20–300kW).
powers_calc	Total input power: Ohmic + auxiliary heating ($\sim$ 1kW–5MW).

It is important to note that TCV shots are generally short in duration. In our dataset, unstable shots have a mean length of 1.6446 s, whereas stable shots have a mean of 1.2848 s. This difference is relevant for the prediction task: accurately estimating the instability onset time is essential, as even small temporal deviations can translate into a significant loss of usable experimental time or failed mitigation action. An example of a typical unstable discharge is shown in Fig. 1.

2.2 Label Construction: Sigmoid-Based Time-to-Instability Encoding

Initially, only the time at which the MARFE onset is experimentally identified is available. However, it is physically expected that the instability develops earlier, over a finite and unknown time interval preceding the observed event. We denote the beginning of this transition as the instability horizon (IH), defined as a time point prior to the MARFE onset at which the plasma starts to enter an unstable regime. Since this horizon cannot be directly measured, it is treated as a tunable parameter. A central objective of this study is therefore to investigate how model predictions depend on the choice of this horizon and to assess which values yield the most accurate and physically meaningful forecasts (see Sec. 4.1).

Figure 2 illustrates the proposed labeling strategy. A traditional step-function labeling assigns binary labels that jump instantaneously from stable (0) to unstable (1) at the chosen horizon. While simple, this approach introduces an artificial discontinuity and implicitly assumes that the plasma state changes abruptly, which is inconsistent with the gradual nature of instability formation. To better

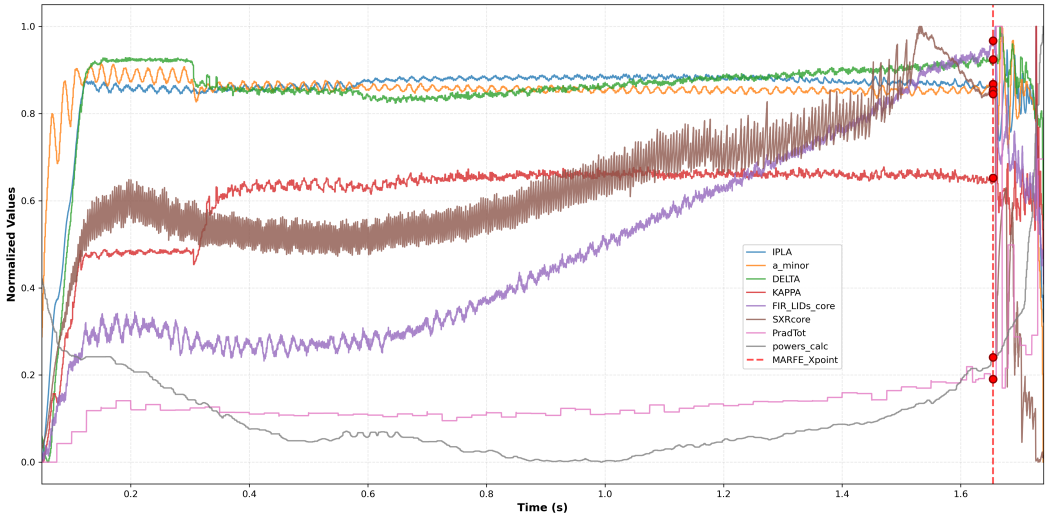


Figure 1: Example of an unstable TCV discharge showing the evolution of the diagnostic features and the MARFE onset time at $t = 1.654\,\mathrm{s}$. Min–max normalization was used for visualization purposes.

reflect the underlying physics, we instead adopt a sigmoid-based soft labeling scheme. The sigmoid smoothly transitions from 0 to 1 over a finite time window, reaching a value of 0.5 at the instability horizon and converging to 1 at the experimentally observed MARFE onset. This formulation provides a continuous approximation of the latent instability probability, capturing both temporal proximity to the event and uncertainty in the onset time. Beyond its physical motivation, the soft-labeling offers practical advantages for learning. It provides richer supervisory signals than hard labels, which can improve optimization stability during training. It is also well suited to models that operate on individual time points rather than entire shots, such as the MLP architecture introduced in Section 3.3.

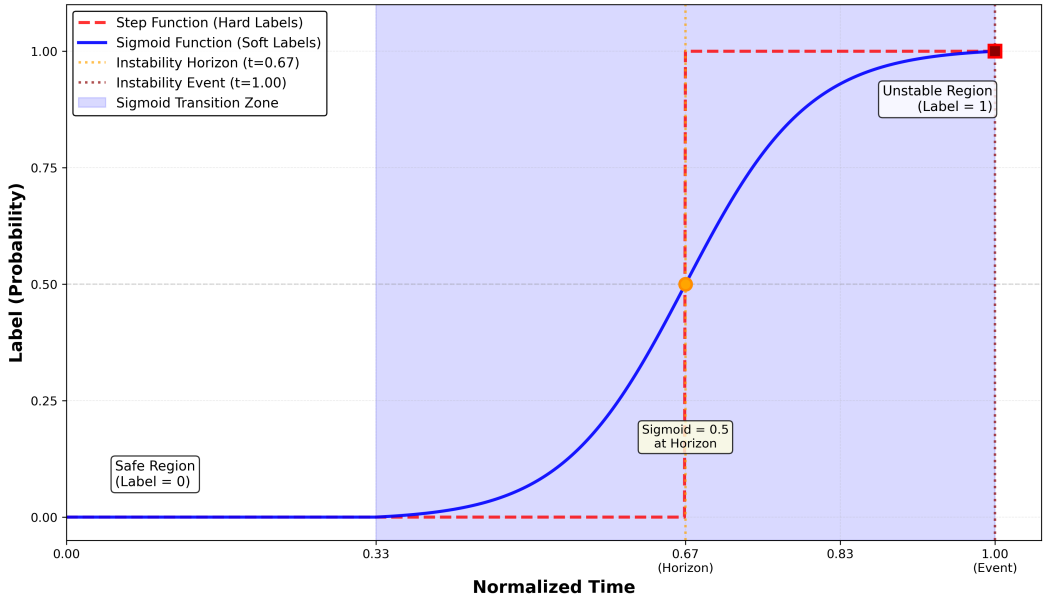


Figure 2: Illustration of the sigmoid-based labeling strategy used to encode the time-to-instability. The experimentally observed MARFE onset occurs at normalized time $t = 1$, while the instability horizon is set at $t = 2/3$, where the sigmoid attains a value of 0.5.

The soft labels are constructed using a bounded sigmoid function. Let $x \in [0, 1]$ denote the normalized time within a discharge, where $x = 1$ corresponds to the experimentally identified MARFE onset. The instability horizon is defined at $x_0 < 1$ and $d = t_{\mathrm{MARFE}} - x_0$ is the distance from the

horizon to the instability. We start from the standard logistic sigmoid:

$$\sigma(x) = \frac{1}{1 + e^{-k(x-x_0)}},$$

which satisfies $\sigma(x_0) = 0.5$ (see Fig. 2). The steepness parameter is defined as $k = \frac{5}{d}$. This choice provides a smooth yet sufficiently sharp transition and could be treated as an additional tunable parameter in future studies. To ensure that the label is close to zero at a time $x_0 - d$ before the horizon and close to one at a time $x_0 + d$ after it, the sigmoid is shifted and rescaled. Defining

$$\sigma_{\min} = \sigma(x_0 - d) = \frac{1}{1 + e^{kd}}, \qquad \sigma_{\max} = \sigma(x_0 + d) = \frac{1}{1 + e^{-kd}},$$

the bounded sigmoid label is given by

$$f(x) = \frac{\sigma(x) - \sigma_{\min}}{\sigma_{\max} - \sigma_{\min}}, \tag{1}$$

followed by clipping to the interval $[0, 1]$ to guarantee that labels are set to $f(x) = 0$ for $x < x_0 - d$ and $f(x) = 1$ for $x \geq x_0 + d = t_{\text{MARFE}}$ (in most shots there is data availability even after the MARFE onset as seen in Fig. 1).

2.3 Standardization and outlier handling

All input features were standardized using z-score normalization, making them more robust to outliers and better suited for gradient-based learning algorithms than min–max scaling (see [7]). For each feature x, the mean μ_{train} and standard deviation σ_{train} were computed exclusively on the training set, and each sample was transformed as:

$$\tilde{x} \;=\; \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}. \tag{2}$$

It is essential that the statistics μ and σ are estimated using only the training data. If the validation and test sets were used for computing these quantities, information from unseen data would implicitly influence the transformation applied during training. This can lead to overly optimistic performance estimates and compromise the validity of the evaluation, since the model would no longer be tested on truly unseen data. We additionally applied outlier clipping at very large deviations from the mean. Before normalization, each feature was clipped to the range $[\mu_{\text{train}} - 100\sigma_{\text{train}},\ \mu_{\text{train}} + 100\sigma_{\text{train}}]$.

3 Models

All models presented in this work were trained on a single NVIDIA Tesla V100-PCIE-32GB GPU.

3.1 Shot-wise classification and evaluation

Although the model outputs a predicted probability at each individual timepoint within a shot, the final objective is shot-wise classification, i.e. determining whether an entire discharge will ultimately lead to an instability. Consequently, predictions are aggregated at the shot level, and performance is evaluated using shot-wise metrics. The primary evaluation scores are precision, recall, and the F_2 score, defined as:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \qquad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \tag{3}$$

$$F_2 = \frac{(1 + 2^2)\,\text{Precision} \cdot \text{Recall}}{2^2\,\text{Precision} + \text{Recall}}. \tag{4}$$

Here, TP, FP, and FN denote true positives, false positives, and false negatives at the shot level. A shot is counted as a true positive in the case where it is disruptive and a valid alarm is raised within the allowed time window (i.e. not earlier than T_{early} if enabled). A false positive is a non-disruptive shot for which an alarm is raised and a false negative is a disruptive shot for which no alarm is raised. A false negative can happen either because the predicted probability never exceeds the detection threshold, or an alarm is raised too early and is explicitly invalidated by the T_{early} constraint. There

are examples of these cases in Fig. 3 and the thresholding logic of how these are determined is explained later in this section and Fig. 4. The F_2 score is selected as the primary performance measure, as it provides a balanced summary of both precision and recall while explicitly assigning greater weight to recall, which is essential for prioritizing the detection of instabilities. This asymmetry is well aligned with the operational objective, where failing to detect an upcoming instability is significantly more critical than triggering a conservative alarm. While false positives are preferable to false negatives in an operational setting, predictions must still be issued as close as possible to the instability horizon in order to preserve valuable operational time. Since a substantial amount of energy is expended during each shot, it is desirable to exploit stable operating conditions for as long as possible before triggering the Disruption Mitigation System (DMS) (see Fig. 3b). We therefore analyze, in Section 4.3, both the anticipation time of correctly predicted shots and the frequency of false positives (see Fig. 3d).

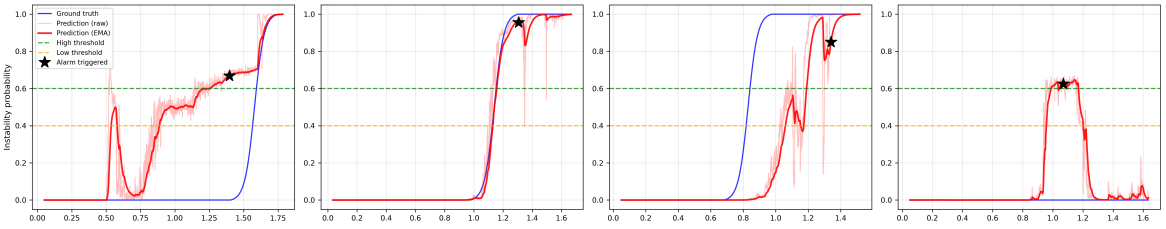


Figure 3: Illustration of the hysteresis-based alarm strategy applied to shot-wise prediction scores. The parameters are set to $T_{\text{high}} = 0.6$, $T_{\text{low}} = 0.4$, $T_{\text{hyst}} = 150\,\text{ms}$, and $\alpha = 8 \times 10^{-3}$. (a) The predicted score drops below T_{low}, causing the hysteresis counter to reset and the alarm is eventually raised shortly before the instability horizon (IH). (b) The alarm is triggered slightly before the onset of the instability, which corresponds to the desired behavior, as it leaves sufficient time for the Disruption Mitigation System (DMS) to activate while fully exploiting the available stable operating time. (c) The alarm is raised too late, after the instability has effectively developed and such cases are considered false negatives, as potential damage to the machine may already have occurred. (d) A false positive case, where an alarm is triggered despite the absence of a subsequent instability.

To convert time-resolved probabilities into a single shot-level decision, we employ a hysteresis-based thresholding strategy rather than a single fixed threshold based (see Fig. 4) . This strategy is adopted from [11] and the `DisruptionBench` alarm-classification class provided by the authors is directly used in our codebase, with minor modifications to adapt it to our data format and evaluation pipeline. The method relies on three parameters: an upper threshold T_{high}, a lower threshold T_{low}, and a persistence time $T_{\text{hysteresis}}$. As predictions evolve in time, an internal counter is increased whenever the predicted disruptivity exceeds T_{high}. If the prediction subsequently decreases but remains within the intermediate band $[T_{\text{low}}, T_{\text{high}}]$, the counter is held constant, reflecting a transient relaxation that does not invalidate the alarm condition. If the prediction falls below T_{low}, the counter is reset, corresponding to a return to a sufficiently safe regime (see Fig. 4 and 3a). An alarm is triggered, and the shot is classified as disruptive, only if the prediction remains persistently above T_{high} long enough for the counter to exceed $T_{\text{hysteresis}}$. Compared to a single-threshold approach, this hysteresis-based mechanism is more robust to short-lived fluctuations and noise and reduces spurious alarms. We also have a T_{early} parameter before which if the alarm is raised it will count as a false positive. This parameter is not considered in the following sections (Secs. 3.3, 3.4, 3.5) and is analysed more in depth in Section 4.3.

To mitigate short-term fluctuations in the predicted probabilities, we apply an exponential moving average (EMA) as a post-processing step. A causal EMA is used, as in deployment the model can only rely on past and current predictions, and any smoothing procedure must therefore avoid the use of future information. The causal EMA is defined recursively as:

$$\tilde{p}_t = \alpha\, p_t + (1 - \alpha)\, \tilde{p}_{t-1}, \tag{5}$$

where p_t denotes the raw predicted probability at time t, $\tilde{p}_t$ the smoothed prediction, and $\alpha \in (0, 1]$ the smoothing coefficient. We set $\alpha = 0.008$, which was found to provide the best compromise between sufficiently reducing high-frequency noise in the predictions and limiting the temporal delay introduced by the filter. This delay arises because the EMA effectively averages over past values, causing rapid increases in the raw prediction to be reflected more gradually in the smoothed signal. As a result, alarms based on the smoothed prediction may be triggered slightly later than those

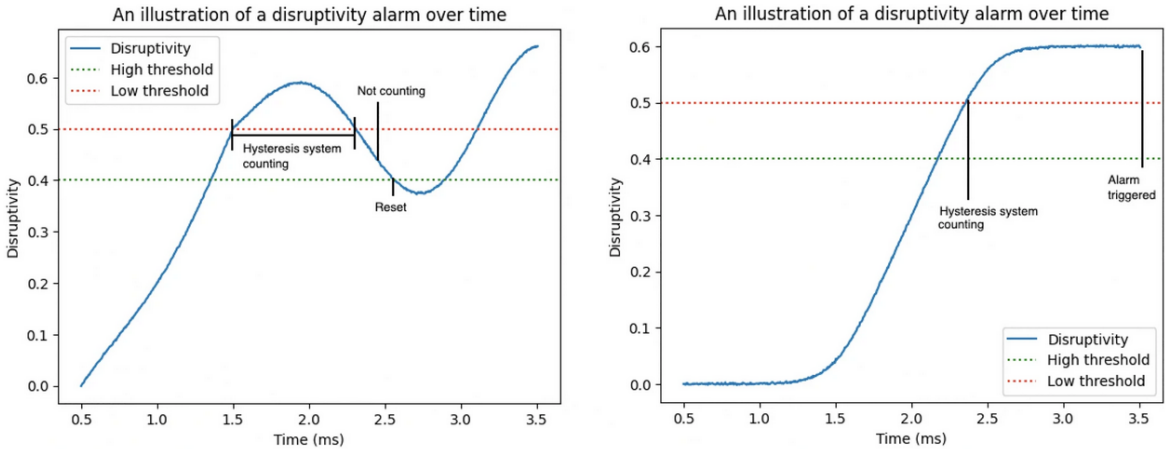


Figure 4: This figure is taken from [11](Fig.1). Two idealized instability predictions unrolled to illustrate binary outcomes in a disrupting warning system with two levels of warnings. The "higher threshold" (defined as T_{high}) is a threshold above which a counter starts to trigger the DMS, whereas the "lower threshold" (defined as T_{low}) resets the counter.

based on raw outputs, an effect that can be observed to a small extent in Fig. 3. In the subsequent hyperparameter analysis (see Figs. 5, 8a, 8b) we use the following values: $T_{\text{high}} = 0.6$, $T_{\text{low}} = 0.4$, $T_{\text{hyst}} = 150\,\text{ms}$ (timepoints) and $T_{\text{early}} = 2$ meaning that if an alarm is raised 2 seconds before the IH then the shot will be counted as a false positive. We delibarately choose a value this high as for the following sections we only care to see if an alarm is raised at all.

3.2 Class imbalance mitigation

The anomaly labeling produces a strongly imbalanced dataset. Using the threshold 0.5 to separate negative and positive labels, we found that 92.5% of training labels are negative and therefore only 7.5% of them are positive. A model trained naively can minimize the loss by over-predicting the majority class. This behavior is problematic because it can lead to deceptively strong aggregate metrics (which was initially noticed in the MSE results) while failing to correctly detect the minority class, resulting in a high false-negative rate. But it is the minority class that is of interest as this is the one encoding the instability.

The first of the two strategies used to address this imbalance, consists in replicating the high-anomaly subset until its size matches the number of low-anomaly samples. The required replication factor is defined as:

$$\lambda = \left\lceil \frac{n_{\text{low}}}{n_{\text{high}}} \right\rceil \tag{6}$$

where n_{low} and n_{high} denote the number of low and high-anomaly samples, respectively. This oversampling was applied only to the neural-network architecture that explicitly processes individual time-points. This is delibarate as when the input is a sequence, it is straightforward to augment the dataset by replicating and reshuffling labeled timepoints within sequences, thereby increasing exposure to minority-class patterns without altering the underlying labeling rule. For TCN and RNN architectures, the temporal structure is part of the signal: the ordering of the timepoints is what allows the model to learn meaningful dynamics. A naive oversampling approach that replicates isolated high-label timepoints (or segments) would introduce artificial repetitions and create spurious correlations as repeated rare events are happening periodically. This would totally bias the training distribution and undermine the model's ability to learn time-dependent behavior.

To address class imbalance for the TCN and RNN models while preserving time dependence, we use a weighted binary cross-entropy loss with logits which has been shown to be effective for learning under class imbalance (see [2], [3]). Let $z \in \mathbb{R}$ denote the model logit for a given timepoint and $y \in [0, 1]$ the target label (interpreted as a probability-like anomaly label). We define the sigmoid function as:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{7}$$

The standard binary cross-entropy (BCE) loss for a single sample is:

$$\ell_{\mathrm{BCE}}(z, y) = -\Big(y \log\big(\sigma(z)\big) + (1 - y) \log\big(1 - \sigma(z)\big)\Big). \tag{8}$$

Standard BCE treats errors on both classes equally, which in a highly imbalanced setting encourages the model to favor the majority (negative) class. Introducing a weight factor λ explicitly increases the cost of misclassifying minority-class samples, thereby aligning the optimization objective with the practical importance of detecting rare events. We introduce a class-dependent weighting based on a threshold $\tau = 0.5$ and define the weight function:

$$w(y) = \begin{cases} \lambda, & \text{if } y \geq \tau, \\ 1, & \text{if } y < \tau. \end{cases} \tag{9}$$

λ is determined using relation 6. The resulting weighted loss over a batch of sequences, with logits and targets of shape $(B, L, 1)$, is given by:

$$\mathcal{L} = \frac{1}{BL} \sum_{b=1}^{B} \sum_{t=1}^{L} w(y_{b,t})\ \ell_{\mathrm{BCE}}(z_{b,t}, y_{b,t}) \tag{10}$$

It is important to note that using BCE with logits avoids an explicit sigmoid in the forward pass and yields better numerical stability, especially when logits have large magnitude, which is common in deep temporal models such as RNNs and TCNs (see [5]). The sigmoid is only applied during the inference to obtain values in $[0, 1]$.

3.3 Multi-Layer Perceptron (NN)

As a baseline that ignores temporal dependencies, we trained a multilayer perceptron (MLP) that predicts an instability score independently at each timepoint. The input to the model is a feature vector $x_t \in \mathbb{R}^d$ extracted at time t (with $d = 8$ features in our configuration), and the network outputs a single scalar logit $z_t \in \mathbb{R}$. The architecture consists of three fully-connected hidden layers with decreasing widths:$d \to 64 \to 32 \to 16 \to 1$ implemented as a stack of linear transformations, ReLU nonlinearities, and dropout regularization. Concretely, the model applies:

$$\mathrm{Linear}(d, 64) \to \mathrm{ReLU} \to \mathrm{Linear}(64, 32) \to \mathrm{ReLU} \to \mathrm{Linear}(32, 16) \to \mathrm{ReLU} \to \mathrm{Linear}(16, 1)$$

with dropout probability $p = 0.2$ at every layer. No sigmoid is applied in the network forward pass, since training is performed with the binary cross-entopy loss with logits as in Eq. 8, which internally combines the sigmoid operation with the binary cross-entropy in a numerically stable manner. The optimizer is Adam, and the default mini-batch size is fixed to 4096. We performed a learning-rate cross-validation by scanning a logarithmically spaced grid of 20 values between 10^{-5} and 10^{-2} for each instability horizon. This procedure was repeated across the five horizons considered in this work: IH $\in \{100, 200, 300, 400, 500\}$ ms. In stochastic gradient-based optimization, the mini-batch size and the learning rate are not independent hyperparameters but are strongly coupled through their effect on the gradient statistics and the effective update magnitude. For a fixed learning rate, reducing the mini-batch size increases the variance of the stochastic gradient estimator, which can lead to noisy updates and instability during training. Conversely, increasing the mini-batch size reduces gradient noise but also decreases the frequency of parameter updates per epoch, effectively changing the optimization dynamics (see [6]). For this reason, performing an independent hyperparameter search over both the mini-batch size and the learning rate would be redundant and significantly increase the search space without providing additional insight. Instead, we fix the mini-batch size to a large value to maximize computational throughput on GPU hardware and conduct a dedicated hyperparameter sweep over the learning rate only (see Fig. 5). No clear monotonic trend with respect to the learning rate can be identified across horizons in Fig. 5, and the performance differences between the two model capacities remain limited, indicating that increasing the number of parameters does not yield a systematic improvement in this setting. A more pronounced trend is observed across instability horizons: shorter horizons consistently achieve higher F_2-scores than longer ones. This trend is analyzed in more detail in Section 4.1.

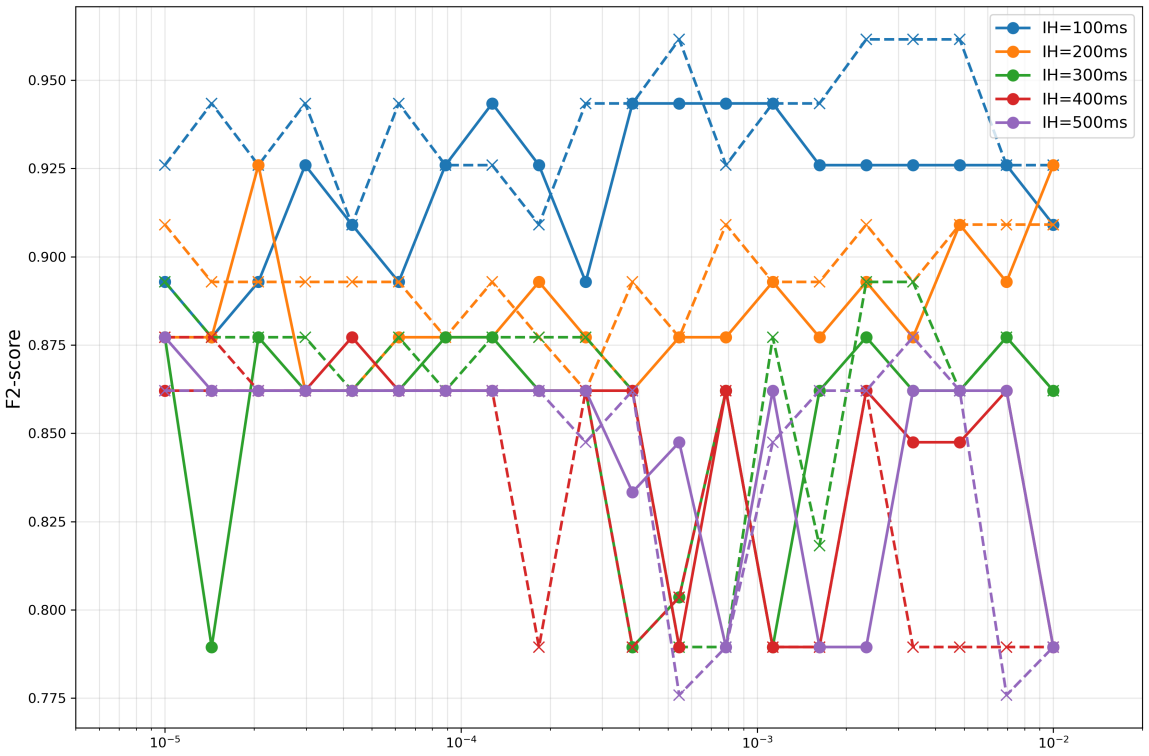


Figure 5: Shot-wise F2-score as a function of the learning rate for different instability horizons (IH). Solid lines correspond to the baseline MLP architecture, while dashed lines represent the same model with doubled hidden-layer widths at each layer.

3.4 Temporal Convolutional Neural Network (TCN)

In the following analysis we use the logic and code proposed in [1]. The central architectural principle in TCN is the use of dilated causal convolutions to efficiently model long-range temporal dependencies while preserving strict causality. The output at time t is allowed to depend only on input samples from the past, ensuring that no future information leaks into the prediction. Dilation provides a mechanism to extend this temporal dependency range without increasing the kernel size or the computational cost excessively. A causal dilated convolution with kernel size k and dilation factor d computes the output at time t according to:

$$y_t = \sum_{j=0}^{k-1} w_j \, x_{t-jd}$$

where w_j are the learnable convolutional weights. When $d = 1$, the operation reduces to a standard causal convolution. For $d > 1$, the kernel taps are spaced d time steps apart, allowing the convolution to cover a larger temporal span while keeping the number of parameters fixed. This mechanism enables the network to capture dependencies over increasingly distant past time steps as the dilation factor grows. To exploit this property, dilated convolutions are stacked in depth with exponentially increasing dilation factors. Specifically, successive layers use dilation values $d = 1, 2, 4, \ldots$, so that each deeper layer aggregates information from a coarser temporal resolution (see Fig. 6a). This strategy allows the temporal receptive field of the network to grow exponentially with depth rather than linearly, which is the key trick behind Temporal Convolutional Networks. The temporal receptive field (RF) of the network is defined as the number of past time steps that can influence a given output time index. For a stack of causal convolutional layers with kernel size k and dilation factors $\{d_1, d_2, \ldots, d_N\}$, the receptive field is given by:

$$\mathrm{RF} = 1 + (k-1) \sum_{i=1}^{N} d_i$$

In the present architecture, each dilation level is implemented as a residual block containing two causal convolutional layers with the same dilation (see Fig. 6b). As a result, the effective receptive

field becomes:

$$\mathrm{RF} = 1 + 2(k-1)\sum_{i=1}^{N} d_i \tag{11}$$

The design objective is to adapt the architecture to a chosen input window length L such that the network achieves full-history coverage, i.e. the receptive field is at least as large as the window length we choose:

$$\mathrm{RF} \geq L$$

Rather than fixing the number of layers in advance, the dilation schedule is constructed incrementally until this condition is satisfied. Using an exponential dilation schedule defined by $d_i = 2^{i-1}$ for $i = 1, \ldots, N$, developing Eq. 11 the receptive field can be written as:

$$\mathrm{RF} = 1 + 2(k-1)\sum_{i=0}^{N-1} 2^i = 1 + 2(k-1)(2^N - 1).$$

For a given window length L, the number of layers required grows only logarithmically with L, making this approach highly efficient for long sequences. From this expression it is also clear that the receptive field can in principle be expanded by increasing the number of network layers N, the dilation rate d, or the kernel size k. However, increasing network depth often leads to the so-called degradation problem, where prediction accuracy saturates and eventually degrades as additional layers are added (see [8], [4]). To address this issue, residual blocks were proposed in this same paper. Residual learning enables information to flow directly from earlier layers to later layers via identity shortcut connections, which has been shown to significantly improve the trainability of deep neural networks. A residual block can be expressed in the general form:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x} \tag{12}$$

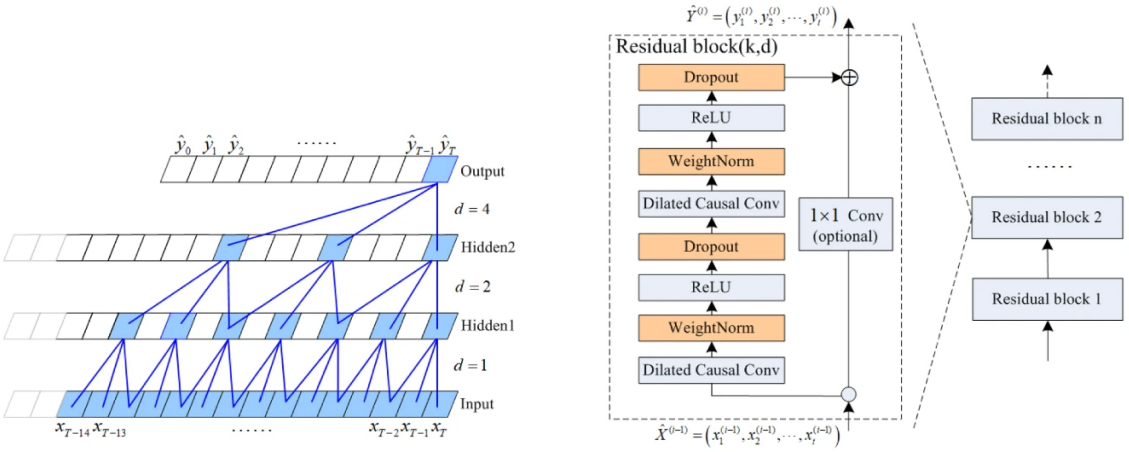
where $\mathbf{x}$ is the block input and $\mathcal{F}(\cdot)$ represents the nonlinear transformation applied within the block. This formulation allows the network to learn residual mappings rather than complete transformations, thereby mitigating gradient degradation (see [8]). In the design of the TCN model, residual blocks are used to replace individual convolutional layers. Each residual block consists of two stacked causal dilated convolutions followed by nonlinear activations and regularization, while the residual connection ensures that information and gradients can propagate efficiently across layers (see Fig. 6b). When the number of feature channels changes between layers, a 1×1 convolution is used on the residual path to preserve dimensional consistency (this is not the case in our implementation as the number of hidden dimensions is fixed). Table 2 gives a detailed summary of the parameters and their exact values in our implementation.

Table 2: Parameters of the implemented Temporal Convolutional Network

Parameter	**Value / Description**
Input window length	$L \in \{128, 256, 512, 1024\}$ (hyperparameter search)
Convolution type	1D causal convolution
Kernel size	$k = 3$
Dilation schedule	Exponential: $d = 1, 2, 4, \ldots$ (determined automatically from L)
Number of residual blocks	Determined automatically from L
Convolutions per block	2 causal dilated convolutions
Feature channels	128
Activation function	ReLU
Learning Rate	10^{-3}
Dropout	$p = 0.2$
Output dimension	One value per time step
Output activation	Linear (logits)
Loss function	Weighted Binary cross-entropy with logits (see Eq. 8)

3.5 Recurrent Neural Network (RNN)

In addition to the temporal convolution, we also consider a sequence-based model relying on recurrent neural networks (RNNs). RNNs are designed to process sequential data by maintaining an internal



(a) Receptive field expansion through stacked dilated causal convolutions. Exponentially increasing dilation factors ($d = 1, 2, 4, \dots$) allow the output at time $\hat{y}_T$ to aggregate information from increasingly distant past inputs while preserving causality.

(b) Temporal Convolutional Network residual block structure. Two causal dilated convolutional layers with identical dilation are combined with a residual connection, enabling stable training and mitigating the degradation problem in deep networks.

Figure 6: Figures taken from [4]. (a) Exponential receptive field growth enabled by dilated causal convolutions. (b) Residual block design used to stabilize optimization when increasing network depth.

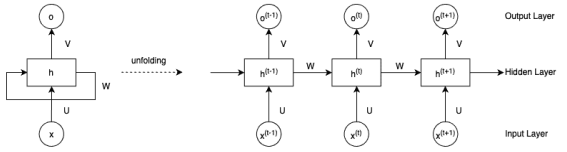
state that evolves over time, making them well suited for modeling temporal dependencies in multivariate time-series data. At each time step t, the model receives an input vector $\mathbf{x}_t \in \mathbb{R}^C$, where C denotes the number of physical input signals, and updates an internal hidden representation $\mathbf{h}_t$ (see 7a) according to:

$$\mathbf{h}_t = f(\mathbf{x}_t, \mathbf{h}_{t-1}) \tag{13}$$

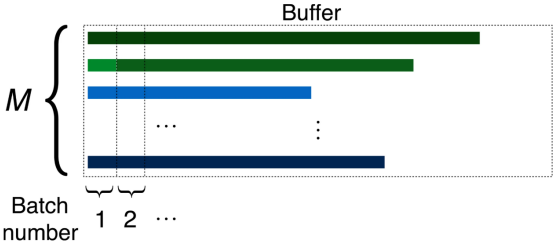
where $f(\cdot)$ is a nonlinear transition function shared across time. The hidden state $\mathbf{h}_t$ acts as a compact memory summarizing past observations and enables the network to capture temporal dependencies. Standard RNNs suffer from the vanishing and exploding gradient problem when trained on long sequences (see [5]). To mitigate this issue, we employ Long Short-Term Memory (LSTM) units, which introduce an explicit memory cell and gating mechanisms that regulate information flow across time. For more details see [5]. The implemented architecture consists of 2 stacked LSTM layers to increase representational capacity. The dropout and the learning rate have the same values as for the TCN model see Tab. 2).

Efficient training on GPUs requires mini-batching but plasma discharges exhibit widely varying sequence lengths, rnaging from 1.2 to 2.3 seconds, which makes direct batching infeasible. An approach such as bucketing is unsuitable because sequence length is correlated with whether a shot is disruptive or non-disruptive, which would therefore introduce bias into individual batches. To address this issue, we implement a stateful, buffer-based training strategy (see [9]). A buffer of size M is maintained, containing M independent shots corresponding to the batch size. At each training step, a fixed-length slice of T_{RNN} consecutive time steps is extracted from each shot in the buffer and fed to the network as a mini-batch. The LSTM internal states are persistent across successive mini-batches for a given shot. After processing one slice, the buffer is advanced by T_{RNN} time steps. When a shot finishes, it is replaced by a new one, and the hidden and cell states corresponding to that batch index are explicitly reset, while the states of the remaining batch indices are preserved. This selective reset mechanism enables batchwise training on sequences of varying lengths while maintaining temporal continuity within each individual shot (see7b). In order for this to work we make sure that the total shot length is a multiople of the selected window value, and if not some timepoints at the beginning are truncated until it is.

In this recurrent framework, the sequence length T_{RNN} plays a role analogous to the receptive field in the Temporal Convolutional Network. It defines the temporal extent over which gradients are explicitly propagated and therefore controls the effective time horizon emphasized during training. A hyperparameter search is performed over different values of T_{RNN}, while keeping the buffering and stateful training mechanism unchanged. From a physical perspective, varying T_{RNN} is meaningful, as it corresponds to probing the characteristic timescale over which plasma history influences instability



(a) Schematic representation of a recurrent neural network (RNN) and its temporal unfolding. The left panel shows the compact recurrent formulation, where the hidden state **h** receives the current input **x** through weights **U** and feeds back into itself through recurrent weights **W**, while producing an output **o** via weights **V**. The right panel illustrates the unfolded representation in time, explicitly showing how the hidden state propagates information across successive time steps, enabling the model to capture temporal dependencies in sequential data.

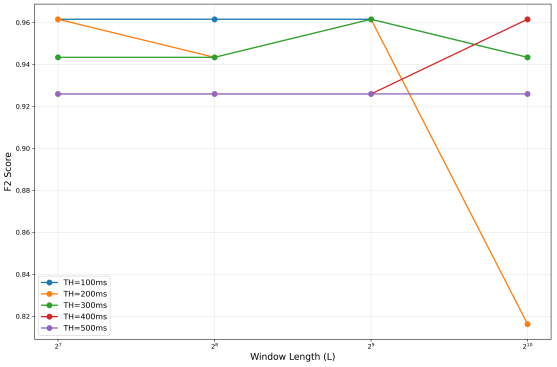


(b) Figure taken from [9]. RNN for batchwise training with a batch size of M. Each horizontal bar represents data from a shot, and different colours indicate different shots. A colour change in a given row means that a new shot starts. At every time step, the leftmost chunk is cut from the buffer and supplied to the training algorithm, and all shots are shifted to the left. When a shot is finished (as the lighter green bar is about to be), a new shot is loaded into the buffer, and the internal state of the RNN at that batch index is reset.

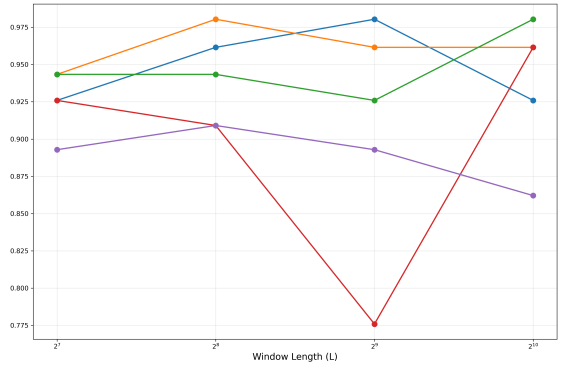
Figure 7

onset.

From Fig. 8a, we observe that the TCNs F2-score remains largely stable across window lengths for most instability horizons, with only limited sensitivity to increases in L. This suggests that once the receptive field covers the relevant temporal scale, extending the input window provides little additional benefit and may even degrade performance for certain horizons. This also holds true for the RNN model (see Fig. 8b). The only exceptions are for the TCN at IH = 200, RF = 2^{10} where the score falls to 0.82, and for the RNN at IH = 400, RF = 2^9 whith a ascore of 0.775.



(a) Shot-wise F2-score as a function of the sequence length for different instability horizons (IH) for the TCN model.



(b) Same for the RNN model.

Figure 8

4 Numerical Experiments

4.1 Cross-Horizon Generalization Analysis

Figure 9 presents the cross-horizon generalization performance of the best-performing models selected from the hyperparameter search for each training horizon. For each model family (MLP, TCN, and RNN), the model trained at a given horizon is evaluated on test datasets spanning all horizons. The F2-score, recall, and precision are reported to emphasize early-warning performance, with recall weighted more strongly through the F2 metric.

A consistent trend is observed for the MLP models: performance degrades monotonically as the test horizon increases. Across all three metrics, MLP models perform best on short horizons (100 and 200), with F2-scores typically in the range 0.75–0.85, recall exceeding 0.85 in some cases, and precision remaining stable around 0.65–0.70. However, a pronounced drop-off is visible for horizons ≥ 300, with F2-scores falling below 0.40 for several trained models and recall decreasing to values as low as $\sim$ 0.25–0.35 at horizon 500. This degradation might suggest that the MLP models struggle to generalize temporal precursors that are further away from the instability onset. From a physical perspective, this translates in stating that the features exploited by the MLP are primarily informative at short timescales, while longer horizons require more explicit temporal modeling. This makes sense at the model works on a timepoint basis and there is no time context. The MLP models trained at 300 and 500 exhibit near-zero performance across all horizons. These models share the smallest learning rate among the hyperparameter configurations, suggesting potential under-training or over-regularization effects. While overfitting cannot be ruled out, the consistently poor generalization indicates that further investigation of optimization dynamics and effective model capacity is required.

The RNN models also exhibit a degradation in performance as the test horizon increases, but the trend is less monotonic and more structured than for the MLP. Notably, RNNs trained at horizons 300 and 400 achieve good results (over 0.8) for small horizons but with a slight dropoff as well at 400 and 500. In contrast, RNNs trained at horizons 100, 200, and 500 generalize significantly worse, with F2-scores dropping below 0.60 for all test horizons. This behavior is somewhat surprising when compared to the MLP results, as one might expect shorter-horizon training to benefit from the temporal modeling capacity of RNNs. Instead, these results suggest that intermediate horizons (300–400) provide a more balanced temporal context.

The TCN models consistently demonstrate the strongest and most robust performance across all training and test horizons. All trained TCN models achieve reasonably high F2-scores, typically above 0.70, with recall often exceeding 0.80 and precision remaining stable around 0.75–0.85. No systematic drop-off is observed as the test horizon increases, in contrast to both NN and RNN architectures. nThis horizon-invariant behavior strongly suggests that the dilated causal convolutions used in the TCN effectively do a better job at capturing multi-scale temporal dependencies that remain informative across a wide range of anticipation times. From a physical standpoint, this could indicate that the TCN is able to exploit temporal patterns in the plasma signals that are present both close to and far from the instability onset. Furhter exploration should be made to understand why there is such a noticeable difference between RNN and TCN.

Overall, these results indicate that model performance is strongly dependent on the temporal structure induced by both the architecture and the training horizon. While MLP and RNN models show clear sensitivity to horizon length, with notable performance degradation at long horizons, the TCN exhibits superior robustness and generalization. TCN should be the model of choice for practical implementation largely due to its robustness.

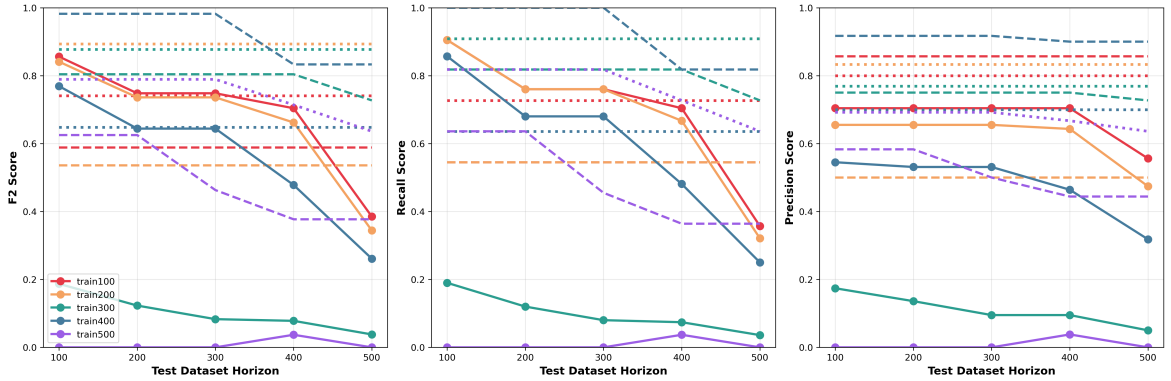


Figure 9: Cross-horizon inference performance of the best models selected from hyperparameter search for each training horizon. For each architecture (MLP, TCN, RNN), the model trained at a given horizon is evaluated on test datasets spanning all horizons. F2-score, recall, and precision are reported. The continuous line is for the MLP models, the dashed line for the RNN ones and the dotted for the TCN.

4.2 Threshold Sensitivity Analysis

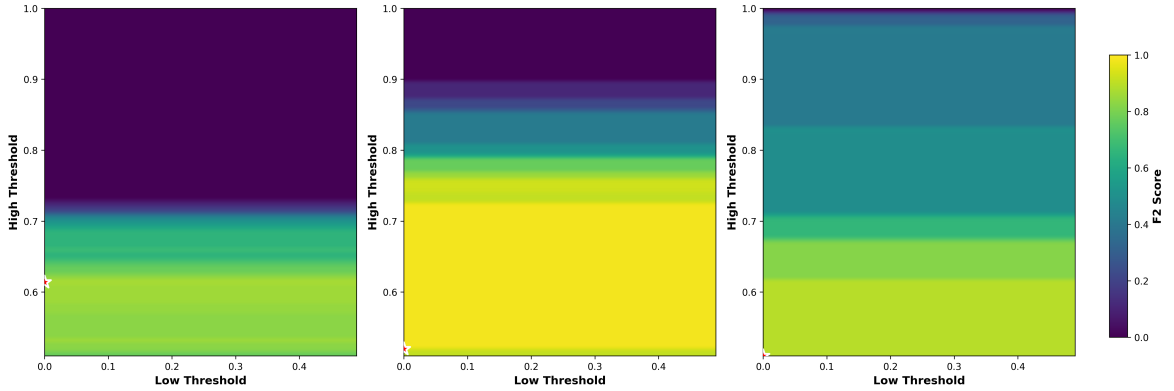


Figure 10: Threshold sweep of the Disruption Mitigation System parameters for the best-performing models selected in the cross-horizon generalization study. From left to right, the heatmaps correspond to the MLP, RNN, and TCN architectures. The F2-score is evaluated as a function of the high disruptivity threshold $T_{\text{high}} \in [0.51, 1.0]$ (vertical axis) and the low disruptivity threshold $T_{\text{low}} \in [0.0, 0.49]$ (horizontal axis). The remaining DMS parameters are fixed, with $T_{\text{hyst}} = 750$ time steps (150 ms) and a small T_{early} to account for all alarms prior to early-warning optimization.

Figure 10 presents a two-dimensional sweep of the disruptivity thresholds (see Fig. 4), applied to the best-performing models identified in the cross-horizon generalization study. From left to right, the heatmaps correspond to the MLP, RNN, and TCN architectures. For each model, the F2-score is evaluated as a function of the high disruptivity threshold T_{high} and the low disruptivity threshold T_{low}, as defined in Sec. 3.1.

The scanned parameter ranges are $T_{\text{high}} \in [0.51, 1.0]$ and $T_{\text{low}} \in [0.0, 0.49]$, ensuring that $T_{\text{low}} < T_{\text{high}}$ at all times. The remaining DMS parameters are fixed: the hysteresis time T_{hyst} is set to 750 time steps (corresponding to 150 ms), and T_{early} is deliberately chosen to be small so that all alarms are counted before optimizing for early warning performance in the next section.

A consistent observation across all three models is that the heatmaps exhibit almost exclusively vertical structure, with negligible variation along the T_{low} axis. This indicates that the low threshold plays a minor role in determining performance in the present configuration, while T_{high} is the dominant parameter controlling the F2-score. This might suggest that there are very few cases in which the predicted disruptivity signal oscillates between T_{high} and T_{low} in a way that would reset the hysteresis counter (see Fig. 4). A plausible explanation is the use of exponential moving average (EMA) smoothing (see Eq. 5) on the model outputs: once the smoothed prediction crosses T_{high}, it tends to remain above this threshold long enough for the hysteresis counter to exceed T_{hyst}, triggering an alarm shortly thereafter. As a result, the mechanism associated with T_{low} consisting in counter reinitialization due to transient drops rarely becomes active. For the rest of the analysis we will solely focus on T_{high}'s behavior. Another direction for future investigation would therefore be to increase T_{hyst}. A longer hysteresis time would make the alarm decision more sensitive to intermediate fluctuations and could allow T_{low} to play a more significant role in suppressing false or premature alarms.

The MLP heatmap exhibits the widest range of relatively low F2-scores, with values starting around 0.7 even in the optimal region. This indicates that the MLP is the most restrictive and least flexible architecture with respect to threshold tuning. Performance improves for moderate values of T_{high}, but the overall plateau remains narrower than for the other models. The MLP relies on sharper decision boundaries and is more sensitive to the precise alarm threshold, consistent with its lack of explicit temporal modeling.

The RNN achieves very strong performance over a broad range of T_{high} values, with F2-scores exceeding 0.8 up to approximately $T_{\text{high}} \simeq 0.8$. Beyond this point, however, the heatmap shows a steep gradient and a rapid transition to very low scores. The RNN predictions cluster strongly below very high disruptivity levels: once T_{high} is set too aggressively, the model fails to trigger alarms altogether. Compared to the MLP, the RNN is thus more powerful but also more brittle with respect to overly strict thresholding.

The TCN displays a smoother and more gradual degradation of performance as T_{high} increases.

While its region of optimal performance is comparable in extent to that of the MLP, the decline in F2-score is significantly less abrupt than for the RNN. Poor performance is observed only for thresholds very close to $T_{\text{high}} = 1$, indicating that the TCN produces more calibrated and temporally consistent disruptivity estimates. This robustness with respect to threshold choice further supports the previous conclusion that the TCN captures physically meaningful multi-scale precursors rather than relying on sharp, high-amplitude excursions (see Sec. 4.1).

4.3 Early-Alarm Penalty Analysis

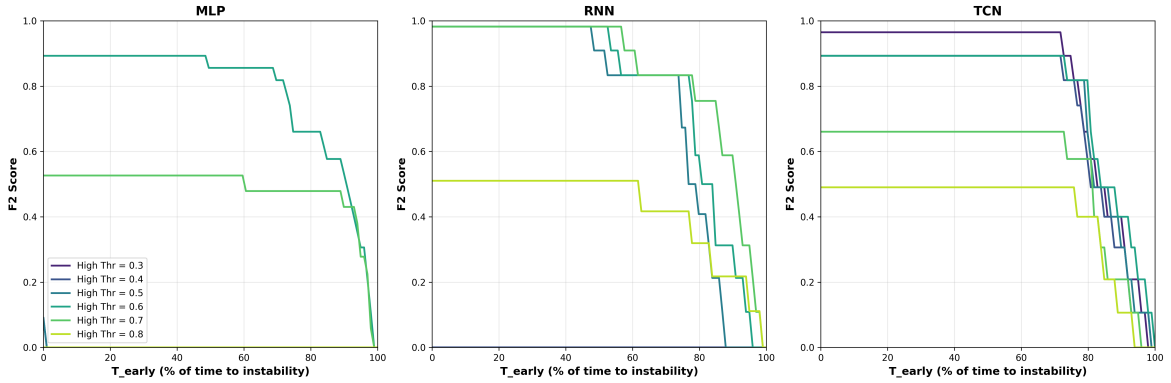


Figure 11: Evolution of the F2 score as a function of the early-alarm penalty parameter T_{early} for the three model families: MLP, RNN, and TCN. Each plot reports six curves corresponding to detection thresholds ranging from 0.3 to 0.8. The parameter T_{early} represents the fraction of the shot duration before the disruption during which alarms are considered too early and are penalized as false negatives.

This analysis provides insight into the temporal behavior of each model, specifically how early alarms are triggered relative to the actual instability and how robust each architecture is to stricter early-warning constraints. We made sure to penalise this early warnings as false negatives instead of flase positives so that the F2 score is penalised more strongly.

The MLP exhibits strong sensitivity to the early-alarm penalty, particularly at low detection thresholds (0.3, 0.4 and 0,5), where the F2 score collapses to zero across almost the entire T_{early} range. This behavior indicates that the MLP frequently raises alarms very early in the shot when using permissive thresholds. Due to the T_{early} logic, these premature alarms are reclassified as false negatives for disruptive shots, leading to a sharp drop in recall and, consequently, in F2 score. As the predicted disruptivity probabilities rarely exceed values of 0.7 or 0.8 for most disruptive shots, when such high thresholds are used, the condition that the predicted probability should be greater than T_{high} is never satisfied during the shot. This results in the MLP having 0 F2-score for the 0.8 threshold and much lower for the 0.7 one. The same trend is noticed for all models. More generally, increasing the detection threshold introduces a fundamental recall–confidence trade-off. Lower thresholds (e.g. 0.3) allows for modest probability excursions to trigger an alarm, resulting in high recall and a greater likelihood of detecting instabilities. However, these alarms tend to occur earlier in the shot and are therefore more likely to be penalized by the T_{early} constraint. Conversely, higher thresholds (e.g. 0.7–0.8) delay or suppress alarm triggering, reducing the risk of early-alarm penalties but increasing the probability of missing instabilities altogether when the model confidence does not reach the required level. The TCN is by far the best performing model in this sense.

The RNN displays a markedly improved behavior compared to the MLP. For intermediate thresholds (notably 0.5 and 0.6), the F2 score remains close to optimal values (near 1.0) up to approximately $T_{\text{early}} \simeq 60\%$ and goes down to 0.8 at $T_{\text{early}} \simeq 80\%$. For these thresholds, the RNN typically raises alarms within the last 20% of the shot before the instability, which is well aligned with the valid early-warning window. However, similar to the MLP, low thresholds lead to near-zero F2 scores due to premature alarm triggering.

The TCN demonstrates the most robust and desirable behavior across all thresholds. Even at relatively low thresholds, the F2 score remains high over a wide range of T_{early}, indicating that alarms are raised naturally closer to the instability time. In particular, the TCN maintains strong performance up to $T_{\text{early}} \simeq 80\%$, meaning that alarms are typically triggered within the final 20%

of the shot. Unlike the MLP and RNN, the TCN does not exhibit a catastrophic failure at low thresholds, suggesting that its predicted disruptivity scores rise more sharply and later in time. It reduces premature threshold crossings and effectively limits early alarms without requiring aggressive threshold tuning.

This analysis highlights clear differences in the temporal alarm behavior of the three architectures. The MLP is prone to very early alarms and lacks temporal precision, resulting in severe penalties under the T_{early} constraint. The RNN significantly improves alarm timing, typically triggering alarms around 50% of the remaining time to instability, but still suffers from threshold sensitivity. The TCN outperforms both alternatives by raising alarms even later, around 70% of the remaining time, providing more usable warning time while allowing the shot to continue safely for longer.

5 Conclusion

The first experiment (see Sec. 4.1) assessed the ability of models trained at a given instability horizon (IH) to generalize across different test horizons. It revealed a strong dependence of performance on temporal modeling capacity. The MLP exhibited a clear degradation in F2 score as the test horizon increased, indicating limited robustness to changes in temporal anticipation. RNNs showed improved generalization, particularly when trained at intermediate horizons, but still suffered from sensitivity to horizon mismatch. In contrast, the TCN consistently maintained high performance across all horizons, suggesting that its dilated temporal convolutions effectively capture multi-scale precursors that are invariant to the specific anticipation window.

The second experiment (see Sec. 4.2) explored the sensitivity of the predictions to the detection thresholds T_{high} and T_{low}. Across all architectures, performance was found to depend almost exclusively on T_{high}, with negligible sensitivity to T_{low}. This behavior is attributed to the smoothness of the predicted disruptivity signals, reinforced by exponential moving average smoothing and a fixed hysteresis time. Among the models, the MLP showed the narrowest region of acceptable thresholds, the RNN achieved high performance but with abrupt failure beyond a critical threshold, and the TCN displayed the most gradual and robust degradation, confirming superior threshold flexibility.

The third experiment (see Sec. 4.3) quantified how model performance degrades as increasingly strict penalties are applied to early alarms through the parameter T_{early}. The MLP was found to trigger alarms too early for low thresholds and to miss instabilities entirely for high thresholds, leading to severe F2 degradation. RNNs demonstrated improved temporal localization, typically raising alarms around 60% of the remaining time to instability, but remained sensitive to threshold choice. The TCN outperformed both alternatives by raising alarms aroundd 70% of the remaining time thereby maximizing usable warning time while minimizing early-alarm penalties. This confirms that the TCN provides the best balance between recall, temporal precision, and operational robustness (see Tab. 3). It is also clear looking at the instability horizon values for the best models in Tab. 3 that shorter instability horizons should be preferred and that probably the underlying physical process of the instability formation happens at shorter time scales.

Table 3: Summary of optimal operating points identified from the numerical experiments for each model architecture.

Model Architecture	IH	Threshold	T_{early} (%)	F2 Score
MLP	100	0.6	50	0.89
RNN	200	0.5	58	0.98
TCN	200	0.3	73	0.97

Beyond the results presented in this study, several directions for further investigation naturally emerge. A first important extension would be to explicitly include the hysteresis time T_{hyst} as a tunable parameter in the optimization process. In the present work, T_{hyst} was fixed to 750 time steps (150 ms), which effectively constrained the interaction between the detection thresholds T_{high} and T_{low}. The threshold sensitivity analysis revealed that, under this configuration, T_{low} has a negligible impact on performance. A systematic sweep over T_{hyst} could reveal regimes in which counter resets become more frequent, potentially increasing the influence of T_{low} and enabling finer control over false and premature alarms. A second important aspect that warrants detailed analysis is the computational cost of each model architecture. While the present study focuses on predictive performance, practical deployment in a real-time disruption mitigation system also requires careful consideration of training time, inference latency, and scalability with respect to available GPU resources. A systematic comparison of training durations, convergence rates, and memory usage together with an evaluation of how these quantities scale with model size, input window length, and batch size would provide valuable insight into the trade-offs between performance and computational efficiency. Such an analysis is particularly relevant when considering deployment on shared HPC infrastructures or real-time control systems with strict latency constraints.

References

[1] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. *An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling*. 2018. arXiv: 1803.01271 [cs.LG]. URL: https://arxiv.org/abs/1803.01271.

[2] Zhongxin Bai et al. "A Weighted Binary Cross-Entropy for Sound Event Representation Learning and Few-Shot Classification". In: *2023 Asia Pacific Signal and Information Processing Association Annual Summit and Conference (APSIPA ASC)*. 2023, pp. 1069–1074. DOI: 10.1109/APSIPAASC58517.2023.10317335.

[3] Zhongxin Bai et al. "End-to-End Speaker Verification via Curriculum Bipartite Ranking Weighted Binary Cross-Entropy". In: *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 30 (2022), pp. 1330–1344. URL: https://api.semanticscholar.org/CorpusID:247644609.

[4] Shuai Chen, Liang Guo, and Lei Ge. "Increasing the Hong Kong Stock Market Predictability: A Temporal Convolutional Network Approach". In: *Computational Economics* 64 (2024), pp. 2853–2878. DOI: 10.1007/s10614-024-10547-y.

[5] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. http://www.deeplearningbook.org. MIT Press, 2016.

[6] Priya Goyal et al. "Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour". In: *arXiv preprint arXiv:1706.02677* (2017).

[7] Jiawei Han, Micheline Kamber, and Jian Pei. *Data Mining: Concepts and Techniques*. 2nd ed. Morgan Kaufmann, 2012.

[8] Kaiming He et al. "Deep Residual Learning for Image Recognition". In: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 770–778. DOI: 10.1109/CVPR.2016.90.

[9] Julian Kates-Harbeck, Alexey Svyatkovskiy, and William Tang. "Predicting disruptive instabilities in controlled fusion plasmas through deep learning". In: *Nature* 568 (2019), pp. 526–531. DOI: 10.1038/s41586-019-1116-4.

[10] U. A. Sheikh et al. "RADCAM—A radiation camera system combining foil bolometers, AXUV diodes, and filtered soft x-ray diodes". In: *Review of Scientific Instruments* 93.11 (Nov. 2022), p. 113513. ISSN: 0034-6748. DOI: 10.1063/5.0095907. eprint: https://pubs.aip.org/aip/rsi/article-pdf/doi/10.1063/5.0095907/16581750/113513_1_online.pdf. URL: https://doi.org/10.1063/5.0095907.

[11] Luca Spangher, Matteo Bonotto, William Arnold, et al. "DisruptionBench and Complimentary New Models: Two Advancements in Machine Learning Driven Disruption Prediction". In: *Journal of Fusion Energy* 44 (2025), p. 26. DOI: 10.1007/s10894-025-00495-2. URL: https://doi.org/10.1007/s10894-025-00495-2.